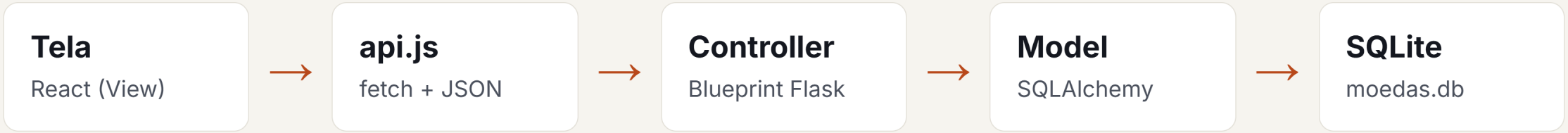
Moeda Estudantil

Arquitetura e camada de persistência

Lucas Gonçalves dos Santos · João Victor

Arquitetura MVC com API REST



Back-end e front-end separados, conversando por **API REST em JSON**. Entregue em sprints, com documentação (casos de uso, classes, ER).

Mapa do back-end

Arquivo	Responsabilidade
app/__init__.py	create_app: banco, CORS, blueprints e seed
app/extensions.py	Instância única do SQLAlchemy
app/models/	Entidades persistentes (ORM)
app/controllers/	Endpoints REST, um blueprint por recurso
app/validacao.py	Validações compartilhadas
app/seed.py	Instituições de ensino iniciais

Decisão: ORM, não DAO com SQL manual

ORM (escolhido)

```
Aluno.query
.filter_by(cpf=cpf).first()
```

DAO + SQL manual

```
SELECT * FROM alunos
WHERE cpf = ?
```

- CRUD sem SQL repetitivo
- Tabelas geradas a partir dos models
- Consultas parametrizadas (contra SQL injection)
- Banco trocável por **DATABASE_URL**

Como o ORM foi montado

```
# extensions.py
db = SQLAlchemy()

# __init__.py: create_app()
URI = os.environ.get("DATABASE_URL", sqlite_moedas_db)
db.init_app(app)
with app.app_context():
  db.create_all()
  seed_data()
```

1. Uma instância de **db** para toda a app

2. Tabelas criadas se ainda não existirem

3. Seed só com a tabela vazia

Modelo ER: o que existe e o que vem

Implementado

usuarios
instituicoes_ensino
alunos
empresas_parceiras

Planejado

professores
vantagens
envios_moeda
trocas_vantagem

Alunos e professores têm relação N:1 com instituições; envios e trocas ligam professor, aluno e vantagem.

Herança com joined table

```
class Usuario(db.Model):
  tipo = db.Column(db.String(20))
  __mapper_args__ = {"polymorphic_on": tipo}

class Aluno(Usuario):
  id = db.Column(db.ForeignKey("usuarios.id"),
      primary_key=True)
  __mapper_args__ = {"polymorphic_identity": "aluno"}
```

usuarios: login, senha_hash, tipo

alunos e **empresas_parceiras**: só campos do perfil

Login único entre todos os perfis

Integridade dos dados

Mecanismo	Onde
UNIQUE: login, e-mail, CPF, CNPJ	Models
NOT NULL nos campos do cadastro	Models
Senha somente como hash	definir_senha (werkzeug)
IntegrityError → rollback + 409	Controllers
CHECK de quantidade e custo > 0	Tabelas planejadas (ER)

Ciclo de um POST /api/alunos

1. Valida JSON e campos (validacao.py)
2. Busca a instituição pelo id
3. Confere e-mail (login) e CPF duplicados: 409
4. Cria Aluno, senha em hash, saldo 0
5. session.add + commit (IntegrityError: rollback)
6. Responde 201 com to_dict()

Testes da persistência

pytest

Testes automatizados do back-end

SQLite em memória

DATABASE_URL definida no conftest; o moedas.db não é tocado

Cobertura

CRUD de aluno e empresa, campos obrigatórios, CPF/CNPJ e unicidade

Limitações e próximos passos

Ponto	Evolução
Controllers falam direto com os models	Camada de serviço para as regras de moedas
create_all() não migra o schema	Flask-Migrate / Alembic
FK no SQLite: PRAGMA não está no ORM	Habilitar foreign_keys e testar
E-mail em usuarios.login e alunos.email	Manter uma fonte única
Professor ainda sem model	Implementar nas próximas sprints